

they'll be called behind the scenes when an object of that class is passed to the `len()` or `repr()` function, respectively. These methods are documented online in the official Python documentation at <https://docs.python.org/3/reference/datamodel.html>.

As we explore the many different types of dunder methods, we'll expand our `WizCoin` class to take advantage of them.

String Representation Dunder Methods

You can use the `__repr__()` and `__str__()` dunder methods to create string representations of objects that Python typically doesn't know how to handle. Usually, Python creates string representations of objects in two ways. The *repr* (pronounced “repper”) string is a string of Python code that, when run, creates a copy of the object. The *str* (pronounced “stir”) string is a human-readable string that provides clear, useful information about the object. The *repr* and *str* strings are returned by the `repr()` and `str()` built-in functions, respectively. For example, enter the following into the interactive shell to see a `datetime.date` object's *repr* and *str* strings:

```
>>> import datetime
❶ >>> newyears = datetime.date(2021, 1, 1)
    >>> repr(newyears)
❷ 'datetime.date(2021, 1, 1)'
    >>> str(newyears)
❸ '2021-01-01'
❹ >>> newyears
datetime.date(2021, 1, 1)
```

In this example, the `'datetime.date(2021, 1, 1)'` *repr* string of the `datetime.date` object ❷ is literally a string of Python code that creates a copy of that object ❶. This copy provides a precise representation of the object. On the other hand, the `'2021-01-01'` *str* string of the `datetime.date` object ❸ is a string representing the object's value in a way that's easy for humans to read. If we simply enter the object into the interactive shell ❹, it displays the *repr* string. An object's *str* string is often displayed to users, whereas an object's *repr* string is used in technical contexts, such as error messages and logfiles.